

Pokhara University
Faculty of Management Studies

Course Code: CMP 172

Course title: **Programming Language**

Nature of the course: Theory + Practical

Year: First, Semester I

Level: Bachelor

Full marks: 100

Pass marks: 45

Credit: 3Hrs

Total periods: 48 hours

Program: BCSIT

1. Course Description

This course is designed to provide students with a comprehensive understanding of computer programming using the C language. Students will develop essential programming skills, ranging from foundational concepts to advanced techniques, enabling them to create efficient, reliable, and maintainable software applications. Through hands-on exercises, coding projects, and problem-solving activities, students will gain practical experience in writing, debugging, and optimizing C programs. The course includes developing the ability to analyze, design, implement, debug, and comprehensively document code within the C programming language. Likewise attaining mastery over pivotal programming concepts in C, encompassing data manipulation and conditional logic. Proficiently employ statements and logical structures for decision-making and program control. Elevate C programming prowess by effectively manipulating arrays and strings, harnessing library functions, crafting custom functions, managing prototypes, and controlling variable scope. Achieving mastery in essential data management skills within C programming, spanning diverse structures, memory optimization, and adept utilization of pointers to enhance efficiency.

2. General Objectives

- To understand the basic principles and concepts of computer programming.
- To gain proficiency in writing C programs and applying programming constructs.
- To develop problem-solving and logic building skills through programming exercises.
- To acquire knowledge of fundamental data structures and their implementation in C.
- To learn how to design and implement algorithms using C programming.
- To apply programming concepts and C programming skills to solve real-world problems.

3. Method of Instructions

General Instructional Technique: Lecture, Discussion, Readings, Question Answer

Specific Instructional Technique: Practical works, Project Based Learning, Self-Directed Learning, Industry Insights and Case Study.

4. Content in Detail with Specific Objectives

Specific Objectives	Content
- To offer students a comprehensive understanding of the C programming language, including its historical context, problem-solving skills, coding techniques, debugging strategies, and best practices for testing and documenting code. - These objectives collectively contribute to building a solid foundation in C programming.	Unit 1: Problem Solving with Computer [6 Hrs.] 1.1 Problem analysis 1.2 Algorithms and Flowchart 1.3 History of C 1.4 Structure of C program 1.4.1 Debugging 1.4.2 Compiling a C program 1.4.3 Executing a C program 1.5 Testing and Documentation 1.6 Preprocessor and Macros
Specific Objectives	Content
- To provide student with a strong foundation in the essential elements of the C programming language. - To student will gain proficiency in constructing C code, manipulating data, and understanding the fundamental building blocks of C programs.	Unit 2: Elements of C [2 Hrs.] 2.1 C Character Set 2.2 Tokens 2.3 Escape sequence 2.4 Variables Declaration and Initialization 2.5 Data types 2.5.1 Basic, Derived, and User Defined 2.6 Constants/ Literals 2.7 Expressions 2.8 Statements 2.9 Comments
Specific Objectives	Content
- To equip learners with the skills necessary to effectively read and write data in C programs. - To learners will be able to interact with users, handle input data, and present	Unit 3: Input and Output [2 Hrs.] 3.1 Unformatted and Formatted I/O 3.2 Conversion specifier 3.3 Character Input and Output

output results in a controlled and organized manner.	3.5 I/O operations
Specific Objectives	Content
- To learners with a deep understanding of various operators and their usage in C programming. - To learners will be able to manipulate data, create complex conditions, and optimize code for efficient execution.	Unit 4: Operators and Expression [4 Hrs.] 4.1 Operator, Operand and Expression 4.2 Types of Operators 4.2.1 Arithmetic operator 4.2.2 Relational operator 4.2.3 Logical or Boolean operator 4.2.4 Assignment Operator 4.2.5 Bitwise operator 4.2.6 Prefix and Postfix Increment and Decrement operator 4.2.7 Ternary operator 4.2.8 Special Operators (size of and comma) 4.3 Evaluation of Expression 4.4 Operator Precedence and Associativity
Specific Objectives	Content
- To provide learners with the skills to control program execution flow based on conditions, implement looping mechanisms for repetitive tasks, and manage loops with break and continue statements. - To learners will be able to write more dynamic and efficient programs that can adapt to different scenarios and perform iterative actions	Unit 5: Control Statement [8 Hrs.] 5.1 Introduction to Control Structure 5.2 Sequential Control Statement 5.3 Decision Making Statement 5.3.1 if, if else, if else if, nested if and switch case 5.4 Iteration (Loop) Statement 5.4.1 for, while, do while, nested loops 5.5 Break and Continue statement
Specific Objectives	Content

▪ To provide learners with a solid understanding of arrays and strings in C programming ▪ To learners will be able to store and process data using arrays, effectively handle textual information through strings, and utilize string library functions for various string operations	Unit 6: Arrays and string [6 Hrs.] 6.1 Introduction to Array 6.2 Declaration 6.3 Memory Representation of Array 6.4 Initialization of Array 6.5 Types of Arrays 6.5.1 Single Dimensional and Multidimensional 6.7 Character Array and Strings 6.8 Reading and Writing Strings 6.9 Null Character 6.10 Library Functions for String handling
Specific Objectives	Content
▪ To equip learners with the skills to create modular, organized, and efficient programs using functions, while also understanding how variable scope and lifetime influence data management and program structure.	Unit 7: Functions [6 Hrs.] 7.1 Introduction to Functions 7.2 Types of Functions 7.2.1 Library Functions 7.2.2 User defined functions) 7.3 Components of Functions 7.3.1 Function prototype 7.3.2 Function call 7.3.3 Function Definition 7.4 Function Arguments and Return Types 7.5 Passing Arrays to Function 7.6 Passing Strings to Function 7.7 Storage Class (auto, register, static, extern) 7.8 Scope of Variables (Local and Global) 7.9 Recursive Function
Specific Objectives	Content

To declare, manipulate, and utilize pointers effectively, understand pointer arithmetic, work with arrays and character strings, use pointers as function arguments, and implement dynamic memory allocation.	Unit 8: Pointers[4hrs] 8.1 Introduction 8.2 The & and * operator, 8.3 Declaration of pointer 8.4 Chain of Pointers 8.5 Pointer Arithmetic 8.6 Pointers and Arrays 8.7 Pointers and Character Strings 8.8 Array of Pointers 8.9 Pointers as Function Arguments 8.10 Dynamic Memory Allocation 8.10.1 malloc() 8.10.2 calloc() 8.10.3 realloc() 8.10.4 free() 8.11 Pass by value, Pass by reference
Specific Objectives	Content
- To provide learners with a deeper understanding of advanced data structures and their usage in C programming - To organize and manage complex data, efficiently pass and manipulate structured data within functions, and use pointers to navigate and modify structure instances	Unit 9: Structure and Union [4 Hrs.] 9.1 Introduction 9.2 Declaration and Initialization of Structure and 9.3 Structure Variable 9.4 Array of structure 9.5 Passing structure to function 9.6 Passing array of structure to function 9.7 Nested Structure 9.8 Pointer to structure 9.10 Union vs Structure
Specific Objectives	Content

▪ To read data from external files or write data to files as a means of input/output operations. ▪ To store and retrieve information persistently, allowing data to be preserved even after the program terminates.	Unit 10: File Handling[6hrs] 10.1 Concept of File 10.2 File Operation Modes 10.3 Opening and Closing of File 10.4 Input Output Operations in File 10.5 Random access in File
--	---

5.Laboratory Work

It builds the foundation on how to write a program using any high-level language. Hence, this course requires a lot of programming practice so that students will be able to develop good logic building and program developing capability which is essential throughout the course.

Some important contents that should be included in lab exercises are as follows:

1. Introduction to C Programming – basic structure of C program.
2. To illustrate the use of Input, Output, Variable and Constants.
3. To illustrate the different operators-Arithmetic, Relational and Logical Operators in C.
4. To illustration of Control Structures (decision making) and processor directives.
5. To illustration of Control Structures (looping).
6. To illustrate the function programming- simple and recursive functions.
7. To illustrate the concept of Pointer and String.
8. To illustrate the function programming- Call by value vs. call by reference.
9. To illustrate the concept of Structure.
10. To illustrate the different file operations.

Note:

1. Motivate students to create small project work integrating all of the above concepts.
2. Each of the above lab session should cover more than 4 hours of practical work.

6. Evaluation system and Student's Responsibilities

Evaluation System

In addition to the formal exam(s), the internal evaluation of a student may consist of quizzes, assignments, lab reports, projects, class participation, etc. The tabular presentation of the internal evaluation is as follows.

External Evaluation	Marks	Internal Evaluation	Weight	Marks
Semester-End examination	50	Theory		30
		Attendance & Class Participation	10%	
		Assignments	20%	
		Presentations/Quizzes	10%	
		Internal Assessment	60%	
		Practical		20
		Attendance & Class Participation	10%	
		Lab Report/Project Report	20%	
		Practical Exam/Project Work	40%	
		Viva	30%	
Total External	50	Total Internal		50
Full Marks: 50 + 50 = 100				

7. Student's Requirements

Each student must secure at least 45% marks separately in both internal assessment and practical evaluation with 80% attendance in the class in order to appear in the Semester End Examination. Failing to get such a score will be given NOT QUALIFIED (NQ) to appear in the Semester-End Examinations. Students are advised to attend all the classes, formal exams, tests, etc., and complete all the assignments within the specified time period.

Students are required to complete all the requirements defined for the completion of the courses.

8. Prescribed Books and References

Prescribed Text Books:

1. Srivastava. (2009). *C - In Depth - 2Nd Revised Edition*. BPB Publications.
 2. Kanetkar, Y. P. (2006). *Let us c*. BPB Publications.
-

Reference Books:

1. Balagurusamy, E. (2019). *Programming In Ansi C, 8Th Edition*. MC Graw Hill India.
 2. Gottfried, B. S. (1996). *Programming with C Ind Adap ed*. Tata McGraw-Hill Education.
 3. Deitel, P. J., & Deitel, H. M. (2016). *C: How to Program*. Prentice Hall.
 4. Kelley, A., & Pohl, I. (1998). *A book on c: Programming in C*. Addison-Wesley Professional.
 5. Mittal, A. (2010). *Programming In C: A Practical Approach*. Pearson Education India.
-